CROSS / PROBLEM 05

Problem 5 — WISE-generated neural-network scheduling on CROSS

CADathlon · `agent_pset` · AI agents allowed · 100 points

1. Introduction

WISE generates packed homomorphic linear transforms, Winograd convolutions and neural-network execution schedules. Its supplied evaluator targets OpenFHE on CPU. CROSS provides CKKS arithmetic on TPUs. Build the connection between them and maximize measured encrypted inference throughput on TPU relative to a well-tuned CPU implementation.

The required output is an executable NN schedule with reproducible evidence. Operation counts and plaintext simulations help select schedules, but only correct encrypted measurements establish performance.

2. Source contract and workloads

The planner is pinned to WISE commit `485655782dc331e6ebb48b159acac8ba924a9933`. Set `WISE_ROOT` to an unchanged checkout of that revision. The loader checks the commit, tracked changes, unexpected Python modules and imported module location. WISE is downloaded as a separate dependency, not copied into the contestant release.

The initial public suite contains these fixed, seeded synthetic networks:

Case	Network	Batch
test1	Dense 8 → 8 → square → 4	1
test2	Dense 32 → 16 → square → 8	8
test3	Dense 784 → 50 → square → 10	1
test4	4×4 grayscale → 3×3 convolution, 2 channels → square → 2×2 average pool → dense 4	1
test5	8×8 grayscale → 3×3 convolution, 4 channels → square → 2×2 average pool → dense 8	8

Case	Network	Batch
test6	16×16 grayscale → 3×3 convolution, 8 channels → square → 2×2 average pool → dense 10	16

Convolution uses stride one and zero padding one. Pooling uses stride two and no padding. Models, weights, seeds, batch sizes and numerical gates are frozen; the scheduler cannot reduce the workload to improve speed. Distinct examples must remain distinct across packed slots and ciphertext blocks. These workloads measure arithmetic and scheduling, not trained classification accuracy.

The full integration target additionally includes WISE's native Winograd path and its pretrained MNIST CNN. Preserve the supplied learned HerPN polynomial, checkpoint, normalization, input selection and all layers. Publish the final schedule and accuracy before making a pretrained-model speed claim. The full nine-layer checkpoint and native Winograd execution are implemented. CROSS and native CPU both pass the complete pretrained network for one image. Repeated one-image CROSS CPU timing has passed; TPU calibration remains incomplete. See `VALIDATION.md` for the scope of each completed experiment and `TPU_TESTING.md` for running the package on another TPU host.

3. Scheduling and encryption interfaces

`workloads.py` constructs fixed public weights and computes an independent PyTorch oracle. `schedule.py` invokes upstream `pack_fc`, `pack_conv2d_toeplitz`, `pack_avgpool2d` and `LinearTransform.pack`. The resulting schedule retains WISE's ciphertext blocks, diagonal arrays, baby-step and giant-step rotations, output folds and biases.

The provided `bsgs` executor shares baby rotations across output blocks and accumulates terms by giant rotation. `direct` evaluates the identical diagonal map without BSGS, providing a scheduling control. Both paths perform encrypted nonlinear squares between linear layers. Optimize caching, batch layout, rotation reuse, fusion, Winograd transforms and TPU execution while retaining the same logical NN. Changes to WISE itself require a reviewed patch and a new explicit planner identity; never silently substitute another upstream revision.

`--convolution tiled-toeplitz` uses WISE tile/channel-gap layouts to reduce pooling permutations. `winograd` executes its native signed input/output transforms and pointwise maps. The pretrained-only `hybrid` option follows WISE's published 4×4 Toeplitz first layer and 2×2 Winograd second layer. The planner verifies every layout against the original PyTorch network.

The optional `--linear-chunk N` control (1–16, default0) fuses small groups of public diagonal products and modular additions with CROSS's existing kernels. The original composite rescale and WISE rotation schedule remain intact. `--cache-mib` sets the public plaintext cache budget. Record these execution controls alongside raw timing samples; they are available on both CPU and TPU. `--rotation-cache-mib` independently bounds retained public rotation key and transform-control entries (default0). It reuses the identical CROSS kernel and keeps the original synchronization barrier. Report cache occupancy separately from whole-process and device memory; this budget is not a total-memory bound.

The CROSS evaluator receives public weights and evaluation keys, with no secret key in its context. Encryption and decryption belong to the client. Input and output ciphertexts must pass the actual-ring security gate. The live Q×P ring must provide at least 128-bit classical security; copying WISE's demonstration `HEStd_NotSet` setting does not satisfy this requirement. Current CPU and TPU comparisons use identical derived rings, scales and depth budgets.

Public schedule verification must agree with the independent NN oracle within absolute error `1e-10`. Decrypted encrypted outputs must have maximum absolute error at most `1e-5` and exact top-1 agreement. No secret-key access, intermediate decrypt/re-encrypt, answer lookup, plaintext replacement or omitted examples is permitted in server computation. The Python harness is a development check; organizers audit submitted evaluator code and run isolated clients and servers.

For the pretrained checkpoint, the public-plan tolerance is `1e-8`; encrypted outputs must have maximum error at most `1e-3`, nRMSE at most `1e-4`, and exact top-1 agreement. The selection is the first requested 1–64 test images in IDX order. Report the sample count explicitly: checking one complete network forward pass does not establish accuracy on all 10,000 test images. The longer CROSS profile uses 46-bit composite scales and a 60-bit first group to retain the trained logits' integer range, with actual Q×P security still enforced.

4. Build and run

Use a CROSS-supported environment with NumPy, SciPy, PyTorch, einops, matplotlib and tqdm. The WISE planner imports these packages; its OpenFHE extension is not needed for CROSS execution.

```
export WISE_ROOT=/path/to/WISE
# From the extracted contestant release, which has no CROSS runtime, also:
# export CROSS_ROOT=/path/to/cadathlon-p1   (this repository's root)
make -C src all reference-test
cd src
python runner.py --case ../test/test1.in --backend cpu --output cpu-test1.json
python runner.py --case ../test/test1.in --backend tpu --output tpu-test1.json
python runner.py --case ../test/test1.in --backend cpu --strategy direct --output cpu-direct-test1.json
python evaluate.py --backend cpu --convolution tiled-toeplitz --output run/cpu-tiled
python evaluate.py --backend tpu --convolution tiled-toeplitz --output run/tpu-tiled
# Complete pretrained checkpoint, initially one cold correctness request:
python evaluate.py --backend cpu --pretrained-batch 1 --convolution tiled-toeplitz --warmups 0 --
repeats 1 --output run/mnist-cpu-cold
# Plaintext checkpoint accuracy and two-image layout checks:
python pretrained_reference.py --batch 2 --full-cleartext-test-set --output mnist-oracle.json
```

Repeat for all six cases in separate fresh processes. The runner refuses an existing output path. Failure reports and partial samples must be preserved. The TPU backend is explicit: CPU fallback cannot earn TPU credit. Use an exclusive TPU allocation; do not terminate another experiment to obtain it.

5. Timing, CPU comparison and scoring

Use at least one warmup and three fresh encrypted requests for each measured configuration. Synchronize device work at every timing boundary. Report planning, key generation/setup, warmup, median server execution, median client-to-client latency, total process wall time and peak host/device memory separately. A single cold request is a correctness experiment, not a calibrated warm speedup. The current runner reports internal elapsed time; collect whole process wall time externally, including exit and cleanup.

Publish CPU model, core allocation, thread settings, TPU model/chip count, software versions, exact runtime sources, schedule digest and security profile. Use the fastest correct CPU schedule among the compared configurations. The initial scorer compares matching CROSS CPU/TPU schedules, permitting different `direct`/`bsgs` execution. A broader claim against CPU HE systems also requires the tuned native WISE/OpenFHE baseline at matched security and accuracy. That baseline is being calibrated; `native_evaluate.py` preserves every thread, ring, precision, scaling and layout attempt in a fresh process. The native runner requests `HEStd_128_classic`; undersized rings are failed attempts. It records the extension and loaded OpenFHE library hashes. See the README for the pinned native build. Record every attempted configuration, including failures; select parameters on public calibration cases before hidden tests.

Component	Points	Evidence
Encrypted correctness	40	All six fixed workloads pass every gate on CPU and TPU
WISE integration and reproducibility	20	Organizer audit of real generated schedules, source/asset identity, client/server boundary and complete resource/timing records
TPU performance	40	`40 × max(0, 1 − 1/S)`, where `S` is the geometric mean of the six CPU/TPU median-server ratios

No performance points are awarded for missing or failed cases. `score.py` requires six CPU and six TPU reports; it checks all warmup/measured samples, source stability and matching schedule/workload/security identity, and recomputes medians from raw samples. Evidence points require organizer review. An optional `--native-cpu` list adds a separate comparison against tuned WISE/OpenFHE. It permits different packing and ring choices while requiring the identical logical workload, pinned WISE sources, enforced 128-bit security, unchanged native binaries and the same accuracy gates. Client diagnostics and incomplete timing records cannot qualify. Native ratios are reported separately from the matched CROSS hardware comparison and its existing point formula. Use `score.py --pretrained` with matching lists of CPU/TPU reports (and optional native CPU reports) to compare distinct pretrained batch sizes. That mode enforces the trained-model error and sample-count gates and reports ratios without awarding synthetic-suite points. For a direct native CPU comparison, `--native-only --native-cpu ... --tpu ...` requires no CROSS CPU reports and awards no contest points. It retains all native/TPU measurement, workload, source, security and accuracy checks, including the six-case requirement for synthetic networks. Add `--pretrained` for one or more distinct complete MNIST batches. This mode reports slower TPU results as well as faster ones; it does not supply the matched CROSS results required for contest scoring or submission. The scored suite and the pretrained/Winograd extension must be reported separately until full calibration is available.

6. Submission

Submit scheduler and backend changes, `cross.patch`, `REPORT.md`, exact WISE revision/patch, all six CPU/TPU result pairs and the complete tuning ledger. Report unfinished experiments explicitly as **not completed**, including the native Winograd path, pretrained-model integration or external CPU comparison if they remain unfinished. `make -C src submit` creates a local archive of the `src/` sources, `cross.patch`, `REPORT.md` and every `results*.json` in this problem directory; copy the result JSON from `src/` and `src/run/` here under that name first (for example `results-tpu-tiled-test1.json`).

---

7. References

- WISE source
- WISE packing and BSGS generation
- CROSS